



## **RTL Design Sherpa**

# **DDR3 / LPDDR3 Family Controller Hardware Architecture Specification 0.6**

**June 13, 2026**

## Table of Contents

|                                                  |    |
|--------------------------------------------------|----|
| 1 Scoria Has Index.....                          | 8  |
| 2 Document Information.....                      | 8  |
| 2.1 Related documents.....                       | 8  |
| 2.2 Terminology.....                             | 9  |
| 2.3 Revision History.....                        | 9  |
| 2.4 How to read this book.....                   | 11 |
| 3 Purpose and Scope.....                         | 12 |
| 3.1 Purpose.....                                 | 12 |
| 3.2 Why a delta specification.....               | 12 |
| 3.3 What is in scope.....                        | 12 |
| 3.4 What is out of scope.....                    | 13 |
| 4 Document Conventions.....                      | 13 |
| 4.1 Block marking.....                           | 13 |
| 4.2 Signal and parameter naming.....             | 14 |
| 4.3 Timing values.....                           | 14 |
| 4.4 Citations.....                               | 14 |
| 5 Definitions and Acronyms.....                  | 15 |
| 5.1 Interfaces.....                              | 15 |
| 5.2 DRAM commands this document refers to.....   | 15 |
| 5.3 Timing parameters new to scoria.....         | 15 |
| 5.4 Terms used with a specific meaning here..... | 16 |
| 6 Scope and Goals.....                           | 17 |

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| 6.1 What scoria is.....                                               | 17 |
| 6.2 Goals.....                                                        | 17 |
| 6.3 Non-goals.....                                                    | 17 |
| 6.4 What makes DDR3/LPDDR3 easier than it looks.....                  | 17 |
| 6.5 What is genuinely harder.....                                     | 18 |
| 7 Block Diagram.....                                                  | 18 |
| 7.1 The whole controller, marked against pumice.....                  | 18 |
| 7.1.1 Figure 2.1: scoria block diagram, with inheritance marking..... | 19 |
| 7.2 Reading the figure.....                                           | 19 |
| 7.3 What the figure deliberately omits.....                           | 20 |
| 8 Module Hierarchy.....                                               | 20 |
| 8.1 Planned hierarchy.....                                            | 20 |
| 8.1.1 Top and macro tier.....                                         | 20 |
| 8.1.2 FUB tier, inherited unchanged.....                              | 21 |
| 8.1.3 FUB tier, modified.....                                         | 21 |
| 8.1.4 FUB tier, new.....                                              | 21 |
| 8.2 Naming, and the package.....                                      | 22 |
| 9 The Target Design Point.....                                        | 22 |
| 9.1 Named as of v0.5.....                                             | 22 |
| 9.2 What this fixes.....                                              | 23 |
| 9.3 The JEDEC timings for this part and bin.....                      | 24 |
| 9.4 What is still open.....                                           | 25 |
| 10 What Is Inherited, and the Blocks That Change.....                 | 25 |

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| 10.1 The inheritance, stated once.....                                          | 25 |
| 10.2 The changes, in one table.....                                             | 26 |
| 10.3 Power-down: smaller than v0.1 thought.....                                 | 27 |
| 11 Init Sequencer: RESET#, MR0-MR3, ZQ Calibration.....                         | 28 |
| 11.1 The sequence is JEDEC's, verbatim.....                                     | 28 |
| 11.2 Three consequences for the sequencer.....                                  | 29 |
| 11.3 ZQ calibration.....                                                        | 29 |
| 11.3.1 Why ZQ gets its own block.....                                           | 29 |
| 12 Write Leveling: the Interface, Not the Search.....                           | 30 |
| 12.1 What write leveling is for.....                                            | 30 |
| 12.2 The decision: scoria provides the interface, firmware runs the search..... | 30 |
| 12.3 What the block owes the system.....                                        | 31 |
| 12.4 The timing windows.....                                                    | 32 |
| 12.5 The telemetry requirement, and why it is not optional.....                 | 33 |
| 13 Refresh: DDR3 All-Bank and LPDDR3 Per-Bank.....                              | 33 |
| 13.1 Marking: INHERITED, with a mode select.....                                | 33 |
| 13.2 What pumice already has.....                                               | 33 |
| 13.3 LPDDR3 keeps the same mechanism.....                                       | 34 |
| 13.4 Refresh by memtype.....                                                    | 34 |
| 13.5 Requirements.....                                                          | 34 |
| 14 DFI v3.1 Master Interface, and the v2.1.1 Delta.....                         | 35 |
| 14.1 The revision decision.....                                                 | 35 |
| 14.2 What does NOT change, and a diff that lies about it.....                   | 36 |

|                                                                        |    |
|------------------------------------------------------------------------|----|
| 14.3 What changes and matters.....                                     | 36 |
| 14.3.1 Leveling: reworked outright.....                                | 36 |
| 14.3.2 Low power: one request becomes two, and nothing consumes either | 37 |
| 14.3.3 Initialization and errors.....                                  | 37 |
| 14.4 What scoria does NOT implement.....                               | 37 |
| 15 AXI4 Slave and APB CSR.....                                         | 38 |
| 15.1 AXI4 slave.....                                                   | 38 |
| 15.2 APB CSR.....                                                      | 38 |
| 15.3 The CDC on the CSR path, and why it is an async FIFO.....         | 38 |
| 16 scoria_pkg, and Build-Time vs Runtime.....                          | 39 |
| 16.1 The package (decision D3).....                                    | 39 |
| 16.2 The rule: timings are runtime, geometry is build-time.....        | 39 |
| 16.3 New CSR groups.....                                               | 40 |
| 17 Verification Strategy, and the Open Questions.....                  | 41 |
| 17.1 The target is the DFI boundary.....                               | 41 |
| 17.2 What must be verified, beyond inheriting pumice's suites.....     | 42 |
| 17.3 Question resolutions (v0.2).....                                  | 42 |
| 17.3.1 Q1 and Q4: I was wrong to call these PHY-blocked.....           | 43 |
| 17.3.2 Q2: ZQCS is a request, never a preemption.....                  | 44 |
| 17.3.3 Q3: sample the prime DQ bit only.....                           | 45 |
| 17.3.4 Q5: struck rather than deferred.....                            | 45 |
| 17.4 What would make this a 1.0.....                                   | 45 |

## List of Figures

|                                                                 |    |
|-----------------------------------------------------------------|----|
| Figure 2.1: scoria block diagram, with inheritance marking..... | 19 |
|-----------------------------------------------------------------|----|

## List of Tables

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| Table 1: Table 0.1: Document information.....                              | 8  |
| Table 2: Table 0.2: Related documents.....                                 | 8  |
| Table 3: Table 0.3: Revision history.....                                  | 9  |
| Table 4: Table 1.1: Block markings.....                                    | 13 |
| Table 5: Table 1.2: Commands, and which are new to this family member..... | 15 |
| Table 6: Table 1.3: New timing parameters.....                             | 15 |
| Table 7: Table 2.1: Top and macro tier.....                                | 20 |
| Table 8: Table 2.2: Modified FUBs.....                                     | 21 |
| Table 9: Table 2.3: New FUBs.....                                          | 21 |
| Table 10: Table 2.4: The target design point.....                          | 22 |
| Table 11: Table 2.5: Build-time parameters fixed by the target.....        | 23 |
| Table 12: Table 2.6: DDR3-800 timings for MT41J256M16.....                 | 24 |
| Table 13: Table 3.1: Every change, with its cause.....                     | 26 |
| Table 14: Table 3.2: DDR3 initialization requirements, from JESD79-3F..... | 28 |
| Table 15: Table 3.3: scoria_wrlvl_ifc responsibilities.....                | 31 |
| Table 16: Table 3.4: Write-leveling timing windows.....                    | 32 |
| Table 17: Table 3.5: Refresh by memtype.....                               | 34 |
| Table 18: Table 4.1: The leveling interface, v2.1.1 against v3.1.....      | 36 |
| Table 19: Table 4.2: DFI v3.1 features deliberately unimplemented.....     | 37 |
| Table 20: Table 5.1: Build-time versus runtime.....                        | 40 |
| Table 21: Table 5.2: New CSR groups.....                                   | 40 |
| Table 22: Table 6.1: Question resolutions.....                             | 42 |

# 1 Scoria Has Index

Generated: 2026-09-30

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

## 2 Document Information

Table 0.1: Document information

| Field        | Value                                                                                         |
|--------------|-----------------------------------------------------------------------------------------------|
| Title        | scoria DDR3/LPDDR3 Family Controller — Hardware Architecture Specification                    |
| Version      | 0.6                                                                                           |
| Date         | 2026-09-29                                                                                    |
| Status       | First edition, <b>pre-RTL</b> . Specifies the controller; does not describe an implementation |
| Scope        | Controller architecture to the DFI v3.1 boundary, for DDR3 and LPDDR3                         |
| Not in scope | The PHY; board bring-up; the DDR4/LPDDR4 features DFI v3.1 also carries                       |

### 2.1 Related documents

Table 0.2: Related documents

| Document                   | Where                           | Relationship                                                      |
|----------------------------|---------------------------------|-------------------------------------------------------------------|
| scoria design requirements | ../design-requirements.md       | <b>binding foundation</b> — the spec deltas and settled decisions |
| pumice HAS                 | ../.. /pumice-ddr2-lpddr2/docs/ | the architecture scoria is                                        |

| Document                   | Where                                                            | Relationship                                                                     |
|----------------------------|------------------------------------------------------------------|----------------------------------------------------------------------------------|
| pumice design requirements | pumice_has/<br>../pumice-ddr2-lpddr2/docs/design-requirements.md | derived from<br>its coding guidelines and enforcement rules apply here unchanged |
| JESD79-3F                  | outside the repo, cold storage                                   | DDR3 device standard                                                             |
| JESD209-3C                 | outside the repo, cold storage                                   | LPDDR3 device standard                                                           |
| DFI v3.1                   | dfi-specs/                                                       | the PHY boundary                                                                 |

## 2.2 Terminology

**INHERITED / MODIFIED / NEW** The marking every block in this document carries, relative to pumice. *Inherited* means the pumice block is used as-is, and its correctness argument transfers with it. *Modified* means a named, bounded change. *New* means no pumice counterpart exists.

**nCK** A DRAM clock cycle, the unit JESD79-3F states most command-spacing minimums in.

**Prime DQ bit** In write leveling, the DQ bit (or bits) on which the DRAM returns the leveling result.

**Maintenance traffic** Commands the controller must issue that no host requested — refresh and ZQ calibration. It competes with demand traffic for the command bus, which makes it a scheduling problem and not only a sequencing one.

## 2.3 Revision History

Table 0.3: Revision history

| Version | Date       | Change                                                                                                                                                                                                         |
|---------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0.6     | 2026-09-30 | Corrected during implementation: dfi_signal_pack is INHERITED, not MODIFIED – it packs only signals DFI v3.1 leaves unchanged, and v3.1's new channels are driven at the layer above. Inherited FUBs 20 -> 21, |

| Version | Date       | Change                                                                                                                                                                                                                                                                                                               |
|---------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0.5     | 2026-09-30 | <p>modified 4 -&gt; 3.</p> <p>Target design point named (Sean: “assume k7ddrphy on genesys2 if it helps”): Genesys 2, K7DDRPHY, 2 x MT41J256M16 on a 32-bit bus, DDR3-800, 3200 MB/s theoretical peak. Fixes DFI_RATE=4, NUM_BANKS=8, row 15, col 10, DQ 32, and gives the DDR3-800 JEDEC timing set.</p>            |
| 0.4     | 2026-09-30 | <p>Q1 and Q4 answered rather than deferred: s7ddrphy is the PHY for every board here, it implements no DFI low-power interface, and it imposes no leveling timeout. powerdown_ctrl corrected to INHERITED in mechanism – power-down is CKE and SRE/SRX, not the DFI low-power channel. No open questions remain.</p> |
| 0.3     | 2026-09-30 | <p>Q2 and D2 re-answered from a generated DDR3 LiteDRAM core instead of by analogy (Sean: “you might need to generate a ddr3 version of litedram”). ZQCS</p>                                                                                                                                                         |

| Version | Date       | Change                                                                                                                                                                                              |
|---------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         |            | confirmed as request/grant sharing the refresher FSM; write leveling confirmed as CSR-only with no state machine.                                                                                   |
| 0.2     | 2026-09-29 | Q1-Q5 resolved or deferred with conditions. refresh_ctrl corrected from MODIFIED to INHERITED: pumice already implements REFpb and the device owns the sequence, so Q5 was malformed and is struck. |
| 0.1     | 2026-09-29 | First edition. Written from the delta analysis, with decisions D1-D3 settled. No RTL exists; every block is specified, not described.                                                               |

## 2.4 How to read this book

Chapter 1 fixes purpose and vocabulary. Chapter 2 is the shape of the controller, with one figure in which every block is colour-marked against pumice — read that figure and you know the size of this project. Chapter 3 is the only chapter with substantial new engineering: the four blocks DDR3 and LPDDR3 actually change. Chapter 4 is the DFI v3.1 boundary and what moved from v2.1.1. Chapter 5 is the package and parameters. Chapter 6 is how it will be verified, and the questions this edition cannot answer.

**A caution particular to a pre-RTL specification.** A document written after the fact can be incomplete; one written before can be *unbuildable*. Where this edition states a structure, it is inherited from a controller that is built and measured. Where it states a timing, it is cited. Where neither is true, it says so in Chapter 6 rather than guessing.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

---

## 3 Purpose and Scope

### 3.1 Purpose

---

scoria is a unified, parameterized memory controller for DDR3 SDRAM and LPDDR3 SDRAM, presenting an AXI4 slave to the host and a DFI v3.1 master to the PHY. It is the third member of a family: pumice covers DDR2/LPDDR2 and is built, measured and running on hardware; a DDR4/LPDDR4 controller is planned.

This document specifies scoria's architecture. Its purpose is to be *locked* before RTL begins, so that the implementation has a fixed target and the PRD — which explicitly waits on a locked HAS — can be written.

### 3.2 Why a delta specification

---

scoria is not a fresh design. The controller architecture it uses is pumice's, and that architecture has been through board bring-up, a read-path rebuild that took reads from 291.7 to 571.3 MB/s, and a formal campaign that found two real bugs. Re-deriving it would discard that evidence.

So this specification is organised as a delta. Every block is marked **INHERITED**, **MODIFIED** or **NEW**, and the marking carries an obligation:

- **INHERITED** means the pumice block is used unchanged, and its verification evidence transfers. If a block is marked inherited and then changed during implementation, the marking is wrong and this document must be corrected — not the marking quietly dropped.
- **MODIFIED** means a bounded change with a named cause, always a clause of JESD79-3F, JESD209-3C or DFI v3.1.
- **NEW** means no pumice counterpart. There are two, and both are small.

### 3.3 What is in scope

---

The controller between the AXI4 slave port and the DFI v3.1 master port: the front end, the scheduler, bank and global timing enforcement, refresh, mode registers and initialization, ZQ calibration, the write-leveling interface, the data paths, and the CSR block.

## 3.4 What is out of scope

**The PHY.** DFI is the boundary, as it is for pumice, and for the same reason: IOB serdes, bitflip and delay-line calibration are FPGA- or process-specific and do not belong in a portable controller. This is a deliberate architectural commitment, not an omission.

**The write-leveling search.** DDR3 adds write leveling, and scoria provides the interface to perform it — the DFI handshake, the MR1 path, the timing windows and the telemetry. The search algorithm itself runs in firmware. This was settled as decision D2 and is argued in Chapter 3.

**The DDR4/LPDDR4 half of DFI v3.1.** Targeting v3.1 brings signals scoria has no use for: `dfi_act_n`, bank groups, chip ID, CA parity and `dfi_alert_n`, Data Bus Inversion, and CA training. They are named here so their absence is a decision on the record rather than an oversight.

**Board bring-up.** Verification is against a DFI bus functional model in simulation. The board target is deliberately deferred.

RTL Design Sherpa · Learning Hardware Design Through Practice · GitHub · Documentation Index · MIT License

## 4 Document Conventions

### 4.1 Block marking

Every block carries exactly one marking, and the block diagram colours match:

*Table 1.1: Block markings*

| Marking   | Colour in Figure 2.1 | Meaning                                                         |
|-----------|----------------------|-----------------------------------------------------------------|
| INHERITED | green                | pumice’s block, used as-is; its verification evidence transfers |
| MODIFIED  | amber                | a bounded change with a cited cause                             |
| NEW       | red                  | no pumice counterpart                                           |

## 4.2 Signal and parameter naming

---

Inherited from the pumice design requirements and not restated here: module naming, UPPER\_CASE parameters, r\_/w\_ internal prefixes, and the per-area clock and reset names. Two points bear repeating because they are load-bearing:

- **Clock and reset names are per area.** In the component tree the controller uses `aclk/aresetn`. An `.i_rst_n(...)` connection against an `rtl/common` module will not elaborate, because those use `clk/rst_n`.
- **No assertions in the RTL.** Properties live in external `formal/` blocks. This is a tool-compatibility rule, not a style preference.

## 4.3 Timing values

---

**Every enforced timing is a runtime CSR, not a parameter.** This is inherited from pumice and was learned there: a timing compiled in cannot be swept, and a controller whose timings cannot be swept cannot be characterized.

Where this document states a numeric timing it is cited to a clause, and the unit is given. `nCK` means DRAM clock cycles. Values that vary by speed bin are named but not tabulated here — they belong to the CSR derivation.

**Note:** JESD79-3F states some minimums without a maximum, most notably `tWLMRD`, whose maximum it declares controller-dependent. Where the spec declines to bound something, this document says so rather than inventing a bound.

## 4.4 Citations

---

A claim traceable to a standard is cited as JESD79-3F (DDR3), JESD209-3C (LPDDR3), or DFI v3.1 with a section number. Those PDFs are **not in the repository** and must not be added to it — they are not redistributable. They live in the operator's cold storage; `dfi-specs/ddr3/index.md` records where and why.

A claim inherited from pumice cites the pumice HAS or its design requirements, both of which are in the repository.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

---

## 5 Definitions and Acronyms

### 5.1 Interfaces

**AXI4** The host-side slave interface. Inherited from pumice unchanged, including the width-gearing converters that let the host bus width differ from the DRAM's.

**DFI** DDR PHY Interface. scoria targets **v3.1** (decision D1). The boundary between controller and PHY, and the same boundary the DV repo's bus functional model drives in simulation.

**APB** The CSR access interface, through a PeakRDL-generated register block.

### 5.2 DRAM commands this document refers to

*Table 1.2: Commands, and which are new to this family member*

| Command                       | Present in DDR2?       | Note                                                               |
|-------------------------------|------------------------|--------------------------------------------------------------------|
| ACT, RD, WR, PRE, REF<br>PREA | yes<br>as PRE with A10 | inherited handling<br>DDR3 gives precharge-all<br>its own encoding |
| MRS                           | yes                    | but DDR3 has MR0-MR3<br>where DDR2 had MR0-<br>MR2 plus EMRS3      |
| ZQCL / ZQCS                   | <b>no</b>              | ZQ calibration, long and<br>short. New in DDR3                     |
| RESET                         | <b>no</b>              | DDR3 adds a hardware<br>RESET# pin                                 |
| SRE / SRX                     | yes                    | self-refresh entry and<br>exit                                     |
| REFpb                         | LPDDR2 has it          | LPDDR3 keeps per-bank<br>refresh                                   |

### 5.3 Timing parameters new to scoria

*Table 1.3: New timing parameters*

| Parameter | Governs                              |
|-----------|--------------------------------------|
| tZQinit   | ZQ calibration during initialization |
| tZQoper   | a long ZQ calibration outside        |

| Parameter | Governs                                                                            |
|-----------|------------------------------------------------------------------------------------|
|           | initialization                                                                     |
| tZQCS     | the short, periodic ZQ calibration                                                 |
| tWLMRD    | delay from entering write-leveling mode to the first DQS pulse                     |
| tWLDQSEN  | delay before the controller may drive DQS, by which point the DRAM has applied ODT |
| tWLO      | write-leveling output delay: when the DRAM returns the result                      |
| tWLOE     | DQ output uncertainty, allowing mismatch across DQ bits                            |

All seven are runtime CSRs. tZQinit, tZQoper and tZQCS also gate command issue alongside tMRD, tMOD and tRFC (JESD79-3F).

## 5.4 Terms used with a specific meaning here

**Maintenance traffic** Commands no host requested: refresh and ZQ calibration. Because it competes for the command bus, it is the scheduler's concern and not only the sequencer's.

**Demand traffic** Reads and writes originating from the AXI4 port.

**Prime DQ bit** The DQ bit (or bits) carrying the write-leveling result back from the DRAM.

**Firmware-driven** Performed by software over the CSR interface rather than by a hardware state machine. Both PHY leveling on pumice's board and scoria's write-leveling search are firmware-driven, deliberately.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

## 6 Scope and Goals

### 6.1 What scoria is

---

A memory controller presenting an AXI4 slave to the host and a DFI v3.1 master to the PHY, supporting DDR3 SDRAM and LPDDR3 SDRAM from one parameterized design.

### 6.2 Goals

---

1. **Reuse the pumice architecture, and its evidence with it.** The front end, scheduler, timing enforcement and data paths are inherited. Reuse here is not laziness; those blocks carry board measurements and formal proofs that a rewrite would discard.
2. **Change only what the standards force.** Four blocks change and two are new. Chapter 3 is that list, and it is short on purpose.
3. **Keep the DFI boundary clean.** No PHY responsibilities migrate into the controller — including the write-leveling *search*, which DDR3 makes tempting and decision D2 rules out.
4. **Every enforced timing a runtime CSR.** Inherited from pumice, and the precondition for characterization.
5. **Verifiable before hardware.** The DFI bus functional model is the target, so the controller can be wrong in simulation where it is cheap.

### 6.3 Non-goals

---

**Performance parity with a vendor controller at DDR3 speeds.** scoria's design point is architectural clarity and measurability. pumice reaches roughly 95% of theoretical peak in both directions on its board, and the same structure should carry, but no throughput target is set in this edition.

**The DDR4 feature set.** DFI v3.1 carries it; scoria does not implement it.

**Multi-rank beyond pumice's parameterization.** NUM\_RANKS is inherited. As with pumice, the practical limit is the board, not the controller.

### 6.4 What makes DDR3/LPDDR3 easier than it looks

---

Three things reduce the work substantially, and it is worth stating them so the project is not over-scoped:

- **LPDDR3 is not a protocol break.** JESD209-3C keeps the 10-bit double-data-rate CA bus that LPDDR2 uses, so the LPDDR3 side is timings and mode registers rather than a new command encoding.
- **pumice was written expecting this.** Its row field is already padded to 18 bits with the comment “DDR3 forward-compat”, and `memtype_e` is an enum rather than a boolean — though a one-bit one, which is why decision D3 gives scoria its own package.
- **DFI v3.1 does not disturb the datapath.** Its multi-phase signalling generalises pumice’s explicit `_p0 . . p3` to `_pN`, and the 1:4 frequency ratio is still defined. `DFI_RATE = 4` carries over untouched.

## 6.5 What is genuinely harder

---

- **Write leveling.** New to this family member, and it spans the controller, the DRAM’s MR1 and the PHY. The interface is specified here; the search is not.
- **ZQ calibration as maintenance traffic.** Periodic ZQCS has to be interleaved with demand traffic, which makes it a scheduling problem.
- **A four-register mode-register set.** MR0-MR3 replaces MR0-MR2 plus EMRS3, and the programming order is JEDEC’s. pumice learned that the hard way: its init order was corrected to JEDEC’s after an EMRS3-first sequence, which was benign but wrong.

RTL Design Sherpa · Learning Hardware Design Through Practice · GitHub · Documentation Index · MIT License

---

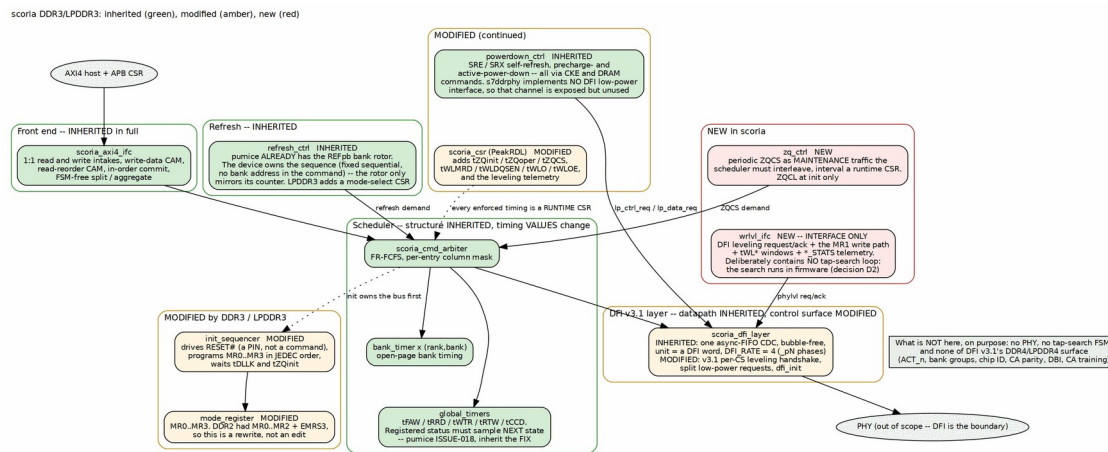
# 7 Block Diagram

## 7.1 The whole controller, marked against pumice

---

Read this figure first. The colour of each block tells you the size of this project: green blocks are pumice’s, used as-is; amber blocks change in named, bounded ways; red blocks are new, and there are two.

## 7.1.1 Figure 2.1: scoria block diagram, with inheritance marking



scoria DDR3/LPDDR3: inherited (green), modified (amber), new (red)

Source: [01\\_block\\_diagram.dot](#)

## 7.2 Reading the figure

**The front end is entirely green.** The AXI4 interface — 1:1 read and write intakes, the write-data CAM, the read-reorder CAM, in-order commit, and the FSM-free split/aggregate — is untouched by the move to DDR3. Nothing about DDR3 or LPDDR3 reaches the host side.

**The scheduler is green in structure, and its timing *values* change.** The FR-FCFS arbiter, the per-(rank, bank) bank timers and the global timers are inherited. What changes is the numbers they enforce, and those are runtime CSRs rather than structure.

**Important:** one inherited detail must be inherited *including its fix*. pumice's `global_timers` published readiness flags one cycle late — registered status sampled the current state instead of the next — so the flags alone permitted violations of tCCD and tRTW. That was pumice ISSUE-018, and the fix was to compute one next-state function and feed both the counter and its status flop from it. scoria inherits the fixed form. A fresh implementation of the same block from the same description would reintroduce the bug.

**The amber blocks are the three FUBs the standards change:** the init sequencer, the mode-register block, and `dfi_cmd_formatter` for the new command encodings — plus the CSR block, which grows to hold the new timings.

**Note:** refresh and power-down are green, and both were amber in earlier editions of this book. Refresh because pumice already implements REFpb and the device owns the sequence (Chapter 3.4); power-down because it works through CKE and SRE/SRX, and the DFI low-power channel that v0.2 built a requirement around is not implemented by this PHY family at all (Chapter 3.1). Both corrections came from reading an artifact instead of reasoning about one.

dfi\_signal\_pack went the same way during implementation: marked MODIFIED for the “v3.1 signal set”, it turned out to pack only signals v3.1 leaves alone.

**The red blocks are the two new ones**, and both are deliberately small: a ZQ controller that issues periodic ZQCS as maintenance traffic, and a write-leveling interface that contains no search loop.

## 7.3 What the figure deliberately omits

The PHY, because DFI is the boundary. A tap-search state machine, because decision D2 puts the search in firmware. And DFI v3.1’s DDR4/LPDDR4 signals, because scoria does not implement them. The note in the figure says so, so that their absence reads as a decision rather than an omission.

# 8 Module Hierarchy

## 8.1 Planned hierarchy

scoria mirrors pumice’s three-tier shape — FUBs under macros under a top — with the scoria\_ prefix replacing pumice\_. The table below is the planned module list with its marking; it is a specification, and the RTL is expected to match it or this document to be corrected.

### 8.1.1 Top and macro tier

Table 2.1: Top and macro tier

| scoria module            | pumice counterpart       | Marking                                         |
|--------------------------|--------------------------|-------------------------------------------------|
| scoria_top               | pumice_top               | INHERITED (structure)                           |
| scoria_top_geared        | pumice_top_geared        | INHERITED — the host/DRAM width-gearing wrapper |
| scoria_core              | pumice_core              | INHERITED                                       |
| scoria_axi4_ifc          | pumice_axi4_ifc          | INHERITED                                       |
| scoria_mem_cmd_scheduler | pumice_mem_cmd_scheduler | MODIFIED — must admit ZQ maintenance demand     |
| scoria_dfi_layer         | pumice_dfi_layer         | MODIFIED — v3.1                                 |

| scoria module | pumice counterpart | Marking         |
|---------------|--------------------|-----------------|
|               |                    | control surface |

### 8.1.2 FUB tier, inherited unchanged

addr\_mapper, bank\_timer, scoria\_bank\_timers, global\_timers (in its fixed form — see Chapter 2.2), scoria\_cmd\_arbiter, scoria\_page\_policy, scoria\_rd\_cmd\_cam, scoria\_wr\_data\_cam, scoria\_rd\_intake, scoria\_wr\_intake, scoria\_wr\_splitter, scoria\_rd\_return\_ring, scoria\_axi\_burst\_chopper, scoria\_dfi\_cdc, scoria\_dfi\_cmd\_path, scoria\_dfi\_rd\_aligner, scoria\_dfi\_wr\_serializer, scoria\_cmd\_history\_checker, and **refresh\_ctrl** — which already carries the REFpb bank rotor for LPDDR2 and needs only a mode-select CSR for LPDDR3, and **powerdown\_ctrl** — power-down is CKE plus SRE/SRX on this PHY family, so only the new command encodings change, and **dfi\_signal\_pack** — a pure registered pipeline stage whose packed signals v3.1 does not touch.

That is 21 of the 24 FUBs carried over with no functional change. The command history checker is worth naming explicitly: it is the verification-side block that independently re-derives JEDEC spacing from the issued command stream, and it must grow DDR3's parameters, but its mechanism is inherited.

### 8.1.3 FUB tier, modified

*Table 2.2: Modified FUBs*

| Module            | Why it changes                                    | Detail |
|-------------------|---------------------------------------------------|--------|
| init_sequencer    | DDR3 adds a RESET# pin and a four-register MR set | Ch 3.2 |
| mode_register     | MR0-MR3 replaces MR0-MR2 plus EMRS3               | Ch 3.2 |
| dfi_cmd_formatter | new command encodings: ZQCL, ZQCS, PREA           | Ch 3.2 |

### 8.1.4 FUB tier, new

*Table 2.3: New FUBs*

| Module         | Purpose                                                                                                                      |
|----------------|------------------------------------------------------------------------------------------------------------------------------|
| scoria_zq_ctrl | issues ZQCS periodically as maintenance traffic; interval a runtime CSR. ZQCL is init-only and belongs to the init sequencer |

| Module           | Purpose                                                                                                                                 |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| scoria_wrlvl_ifc | the write-leveling interface: DFI leveling handshake, MR1 write path, tWL* window enforcement, *_STATS telemetry. <b>No search loop</b> |

## 8.2 Naming, and the package

Per decision D3, scoria gets its own `scoria_pkg` with a one-bit memtype enum covering {DDR3, LPDDR3}. It does not extend `pumice_pkg`, whose `memtype_e` is one bit wide and already spent on {DDR2, LPDDR2} — widening it would change a CSR in a design that is measured and shipping, for a controller with no RTL.

When the DDR4/LPDDR4 controller begins, a shared `mem_ctrl_pkg` with a two-bit memtype becomes worth the migration, and both existing controllers move to it together. That is the right time because the cost is paid once with three members to amortise it, rather than twice.

**Note:** this means `scoria_pkg` and `pumice_pkg` will contain near-identical type definitions for a period. That duplication is deliberate and time-boxed, and it is recorded here so it is not mistaken for an oversight and “fixed” by someone widening `pumice`’s CSR.

# 9 The Target Design Point

## 9.1 Named as of v0.5

Sean, 2026-09-30: “Assume `k7ddrphy` on `genesys2` if it helps.” It helps considerably — it converts most of this specification’s build-time parameters from open to fixed, and it gives every future bandwidth number a denominator.

The board decision for *verification* is unchanged: `scoria` is verified against a DFI bus functional model in simulation. What follows is the **target** the parameters are chosen for, not a claim that a bitstream exists.

Table 2.4: The target design point

| Item  | Value              | Source                   |
|-------|--------------------|--------------------------|
| Board | Digilent Genesys 2 | already on the bench for |

| Item                    | Value                                        | Source                                                          |
|-------------------------|----------------------------------------------|-----------------------------------------------------------------|
| FPGA                    | Xilinx Kintex-7                              | STREAM and RAPIDS board                                         |
| PHY                     | s7ddrphy.K7DDRPHY, memtype="DDR3", nphases=4 | litex-boards<br>digilent_genesys2                               |
| DRAM                    | 2 x MT41J256M16, 32-bit bus                  | platform: 32 DQ, 4 DQS pairs, 4 DM                              |
| Geometry                | 8 banks, 32768 rows, 1024 columns            | LiteDRAM MT41J256M16                                            |
| Address pins            | 15 address, 3 bank                           | platform; matches 2 <sup>15</sup> rows and 2 <sup>3</sup> banks |
| System clock            | 100 MHz                                      | litex-boards default for this target                            |
| Gear ratio              | 1:4                                          | MT41J256M16(sys_clk_freq, "1:4")                                |
| DRAM clock              | 400 MHz                                      | 4 x sys                                                         |
| Data rate               | 800 MT/s – <b>DDR3-800</b>                   | 2 x CK                                                          |
| <b>Theoretical peak</b> | <b>3200 MB/s</b>                             | 800 MT/s x 32 bit / 8                                           |

**For comparison, pumice’s peak is 600 MB/s** (300 MT/s on a 16-bit bus). scoria’s target is 5.3x that, from a wider bus and a higher rate — worth stating plainly because it changes what the front end has to sustain, not just what the DRAM can do.

**Important — 3200 MB/s is the denominator.** The house rule is that a bandwidth table carries the measured figure *and* the theoretical maximum. For scoria on this target the maximum is 3200 MB/s, and a percentage quoted without it is not a result. pumice learned this twice: once by publishing percentages that were understated 2x because the byte counter was per-phase, and once by quoting a figure with no peak beside it.

## 9.2 What this fixes

Build-time parameters that earlier editions left open now have values:

*Table 2.5: Build-time parameters fixed by the target*

| Parameter | Value | Why it is forced                              |
|-----------|-------|-----------------------------------------------|
| DFI_RATE  | 4     | must equal the PHY’s nphases, which is 4. Not |

| Parameter    | Value | Why it is forced                                                                                                        |
|--------------|-------|-------------------------------------------------------------------------------------------------------------------------|
|              |       | a choice, and a mismatch is a functional break that presents as data corruption                                         |
| NUM_BANKS    | 8     | DDR3 geometry                                                                                                           |
| row width    | 15    | 32768 rows. pumice's package already pads the row field to 18 bits<br>“DDR3 forward-compat”, so this fits with headroom |
| column width | 10    | 1024 columns                                                                                                            |
| DQ width     | 32    | board wiring, 2 devices in parallel                                                                                     |
| NUM_RANKS    | 1     | single rank per device, both in parallel on one chip select                                                             |

**DFI\_RATE = 4 is the happy accident worth noting:** pumice also runs four DFI phases, so the inherited DFI datapath — the async-FIFO CDC, the bubble-free pipeline, the per-phase `_pN` signalling — transfers at the same gear ratio it was built and measured at. Had the target been 1:2, the datapath would still work but none of pumice's phase-level measurements would carry over.

### 9.3 The JEDEC timings for this part and bin

From LiteDRAM's MT41J256M16 definition, DDR3-800 speed bin. These are the values the CSRs are initialized to; they remain **runtime** registers.

*Table 2.6: DDR3-800 timings for MT41J256M16*

| Parameter      | Value                    |
|----------------|--------------------------|
| tREFI          | 7812.5 ns (64 ms / 8192) |
| tRP, tRCD, tWR | 13.1 ns each             |
| tRAS           | 37.5 ns                  |
| tRFC           | 139 ns                   |
| tFAW           | 50 ns                    |

| Parameter | Value              |
|-----------|--------------------|
| tRRD      | max(6 nCK, 10 ns)  |
| tWTR      | max(4 nCK, 7.5 ns) |
| tCCD      | 4 nCK              |
| tZQCS     | max(64 nCK, 80 ns) |

**Note:** several are stated as max(nCK, ns), which is JEDEC's form and the reason the CSR derivation must take the larger of the two after converting at the operating frequency. pumice's timing derivation was corrected once for exactly this — a formula that matched one measured point but took the wrong branch elsewhere.

At CK = 400 MHz (2.5 ns), the write-leveling windows of Chapter 3.3 become concrete: tWLMRD 40 nCK = 100 ns minimum, tWLDQSEN 25 nCK = 62.5 ns.

## 9.4 What is still open

Nothing in the specification. Two things remain *choices to be made during implementation* rather than open questions:

- **tWLMRD's maximum**, which is scoria's to define as a timeout (Chapter 6). With CK at 2.5 ns a generous bound is easy to pick; the requirement is that it exists and reports distinctly, not that it be tight.
- **Whether a bitstream is ever built for this target.** The design point is useful for fixing parameters whether or not it is, and verification does not depend on it.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

# 10 What Is Inherited, and the Blocks That Change

## 10.1 The inheritance, stated once

Twenty-one of pumice's twenty-four FUBs carry over with no functional change, as do the top, core, AXI4 macro and the DFI datapath. Chapter 2.3 lists them. This chapter covers only what changes, and the list is short by design.

**The inheritance is an argument, not a convenience.** Those blocks have been through board bring-up, a read-path rebuild that took reads from 291.7 to 571.3 MB/s, and a formal campaign across nine modules. Re-deriving them would discard that evidence and reintroduce the bugs it caught.

**Important — inherit global\_timers in its FIXED form.** pumice’s version published readiness one cycle late: the registered status flop sampled the current counter state rather than the next, so the flags alone permitted tCCD and tRTW violations. The fix computes one next-state function and feeds both the counter and its status flop from it. A fresh implementation written from a behavioural description of the same block would reintroduce the defect, because the description does not mention the flop. Copy the module.

## 10.2 The changes, in one table

*Table 3.1: Every change, with its cause*

| Block             | Marking   | Cause                                                                                                                              | Where |
|-------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------|-------|
| init_sequencer    | MODIFIED  | RESET# is a pin;<br>MR order is MR2-<br>MR3-MR1-MR0;<br>ZQCL closes init                                                           | 3.2   |
| mode_register     | MODIFIED  | MR0-MR3 replaces<br>MR0-MR2 plus<br>EMRS3                                                                                          | 3.2   |
| dfi_cmd_formatter | MODIFIED  | new encodings:<br>ZQCL, ZQCS, PREA                                                                                                 | 3.2   |
| scoria_zq_ctrl    | NEW       | periodic ZQCS is<br>maintenance<br>traffic                                                                                         | 3.2   |
| scoria_wrlvl_ifc  | NEW       | DDR3 adds write<br>leveling                                                                                                        | 3.3   |
| refresh_ctrl      | INHERITED | pumice already<br>implements<br>REFpb; LPDDR3<br>uses the same<br>device-fixed<br>mechanism. Only<br>a mode-select CSR<br>is added | 3.4   |

| Block                    | Marking   | Cause                                                                                                             | Where  |
|--------------------------|-----------|-------------------------------------------------------------------------------------------------------------------|--------|
| powerdown_ctrl           | INHERITED | mechanism unchanged: CKE + SRE/SRX. The DFI low-power channel is unused on this PHY family                        | below  |
| dfi_signal_pack          | INHERITED | every signal it packs is identical in v2.1.1 and v3.1 for DDR3; v3.1's new channels are driven at the layer above | Ch 4.1 |
| scoria_mem_cmd_scheduler | MODIFIED  | must admit ZQ demand alongside refresh                                                                            | 3.2    |
| scoria_csr               | MODIFIED  | new timing registers and leveling telemetry                                                                       | Ch 5   |

## 10.3 Power-down: smaller than v0.1 thought

DDR3 and LPDDR3 both offer precharge-power-down, active-power-down and self-refresh (SRE / SRX). pumice's powerdown\_ctrl already implements idle-timer-driven entry and exit, and **that is the whole mechanism** on a Series-7 PHY target.

v0.1 through v0.3 of this document said the interesting change was that DFI v3.1 splits dfi\_lp\_req into dfi\_lp\_ctrl\_req and dfi\_lp\_data\_req, and left the exit-ordering as an open question. The split is real, but it is not load-bearing here: **s7ddrphy implements no DFI low-power interface at all**, and LiteDRAM's generated DDR3 controller does not drive one either — zero dfi\_lp hits in the generated core against 92 references to CKE.

Power-down on this PHY family is a **DRAM command** matter: CKE, plus SRE and SRX for self-refresh. powerdown\_ctrl is therefore INHERITED in mechanism, and what it gains for DDR3 is the new self-refresh command encodings rather than a new interface.

**Requirement for the DFI low-power ports.** scoria still exposes dfi\_lp\_ctrl\_req and dfi\_lp\_data\_req, because DFI v3.1 defines them and a future PHY may consume them. Their

behaviour when no acknowledgement ever arrives is specified: **time out and report; never block power-down**. An unacknowledged request must not be able to wedge the controller — which is the real hazard, and a smaller one than the ordering puzzle v0.2 imagined.

## 11 Init Sequencer: RESET#, MR0-MR3, ZQ Calibration

### 11.1 The sequence is JEDEC's, verbatim

JESD79-3F specifies DDR3 power-up initialization as a numbered sequence. It is reproduced here as *requirements on the sequencer*, in the spec's order, because the order is not ours to choose:

*Table 3.2: DDR3 initialization requirements, from JESD79-3F*

| Step | Requirement                                                                                                                                                            |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Power ramp, with the spec's VDD / VDDQ / VTT / Vref ordering constraints. Board-level, not controller-level                                                            |
| 2    | After RESET# is de-asserted, wait <b>500 us</b> before CKE becomes active. The DRAM initializes internally during this, independent of external clocks                 |
| 3    | Clocks stable for at least <b>10 ns or 5 tCK, whichever is larger</b> , before CKE goes active. A NOP or Deselect must be registered before CKE goes high, meeting tIS |
| 4    | CKE must be <b>continuously registered high</b> from that point until initialization finishes — including expiry of tDLLK and tZQinit                                  |
| 5    | Wait tXPR = max(tXS, 5 x tCK) after                                                                                                                                    |

| Step | Requirement                          |
|------|--------------------------------------|
|      | CKE high before the first MRS        |
| 6    | MRS to <b>MR2</b>                    |
| 7    | MRS to <b>MR3</b>                    |
| 8    | MRS to <b>MR1</b> , with DLL enabled |
| 9    | MRS to <b>MR0</b> , with DLL reset   |
| 10   | <b>ZQCL</b> to start ZQ calibration  |
| 11   | Wait for both tDLLK and tZQinit      |
| 12   | Ready for normal operation           |

## 11.2 Three consequences for the sequencer

**RESET# is a pin, not a command.** This is the structural change. pumice’s init sequencer issues commands; scoria’s must additionally drive a reset output and hold it through a timed window. That output has to reach the top level and the PHY, and it is the first thing in the sequence rather than something the command path can express.

**The MR order is MR2, MR3, MR1, MR0** — and it is not sorted. Steps 6 through 9 are explicit, and MR1 carries “DLL enable” while MR0 carries “DLL reset”, so the order encodes a dependency rather than a preference.

**Note:** pumice hit precisely this. Its init programmed EMRS3 before EMRS2; the error was benign in practice but wrong, and it was corrected to JEDEC’s order. DDR3’s MR2-MR3-MR1-MR0 is the same pattern extended to a four-register set, and the same rule applies: the sequence is a citation, not a design choice.

**ODT is constrained during init.** The DRAM holds on-die termination in high-impedance while RESET# is asserted and until CKE is registered high; thereafter ODT must be held statically — and **statically LOW if RTT\_NOM is to be enabled in MR1** — until initialization completes. The sequencer therefore owns ODT during init, not just during traffic.

## 11.3 ZQ calibration

ZQCL closes initialization (step 10) and tZQinit gates readiness (step 11). Outside init, DDR3 offers ZQCS, the short calibration, and tZQoper for a long one. All three gate command issue alongside tMRD, tMOD and tRFC.

### 11.3.1 Why ZQ gets its own block

ZQCL at init belongs to the sequencer. Periodic ZQCS does not, because it is **maintenance traffic**: the controller must issue it on an interval no host asked for, competing with demand traffic for

the command bus. That is the same shape as refresh, and refresh has its own controller for the same reason.

`scoria_zq_ctrl` therefore:

- holds the ZQCS interval as a **runtime CSR**, so it can be swept;
- raises a demand to the scheduler rather than seizing the bus;
- enforces  $\tau_{ZQCS}$  before releasing the bus back;
- reports a count of issued calibrations, so the interval can be confirmed from the host rather than assumed.

**The scheduler change is one input, not a rework.** `scoria_mem_cmd_scheduler` already arbitrates refresh demand against demand traffic; ZQ adds a second maintenance source with the same shape.

**Note:** whether ZQCS should ever *preempt* queued demand traffic, or only fill gaps, is deliberately not fixed in this edition. The baseline requirement is that periodic ZQCS is issued at all, with the interval observable and settable. Cleverness about placement is a characterization question, and scoria TASK-001 already owns the survey of scheduling modes.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

---

## 12 Write Leveling: the Interface, Not the Search

### 12.1 What write leveling is for

---

On a fly-by command/address topology, DQS arrives at each DRAM at a different time relative to CK. Write leveling lets the controller discover that skew per device: the DRAM is put into a leveling mode via MR1, the controller drives a DQS edge, and the DRAM reports on its prime DQ bit whether that edge arrived before or after the CK rising edge. The controller walks its DQS delay until the reported value flips. That flip is the answer.

### 12.2 The decision: scoria provides the interface, firmware runs the search

---

This was settled as decision D2. `scoria_wrlvl_ifc` contains **no search loop**.

The reasoning is the family's standing architectural boundary. A delay-line walk is PHY calibration: the step size, the number of taps, their monotonicity and their temperature behaviour

are all properties of the PHY, not of DDR3. pumice made the same call and levels its board from firmware over a CSR passthrough, with no hardware state machine — thirteen PHY knobs reached indirectly, the search in a host program, the settled result cached.

The alternative — a hardware FSM — buys independence from a host at the cost of importing PHY-specific search logic into a portable controller, and of a calibration bug being a silicon respin instead of a script edit.

**Important:** this is a boundary decision, not an efficiency one. A future system that needs autonomous leveling should add an external calibration engine driving this interface, not a search loop inside the controller.

**Corroborated by LiteDRAM.** A DDR3 LiteDRAM core was generated to check this (bin/litedram\_ddr3\_ref.yml). Its write-leveling support is two CSR fields – `wlevel_en` to enter the mode and `wlevel_strobe` to emit a DQS pulse – and **no state machine whatsoever**: a search for leveling FSM state in the generated core returns nothing. Software enables the mode, pulses the strobe, reads the result and walks the delay.

An independent and widely-deployed controller reaching the same conclusion is worth more than the argument above on its own. It also tells us the *shape* of the interface that works in practice: enable plus strobe, rather than a higher-level “do a leveling pass” command.

## 12.3 What the block owes the system

Table 3.3: *scoria\_wrlvl\_ifc* responsibilities

| Responsibility         | Detail                                                                                                                                                                             |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DFI leveling handshake | drive <code>dfi_phylvl_req_cs_n</code> and observe <code>dfi_phylvl_ack_cs_n</code> , with <code>dfi_phy_wrlvl_cs_n</code> selecting write leveling; per chip select, per DFI v3.1 |
| MR1 path               | enter and leave the DRAM’s write-leveling mode by MRS to MR1, sequenced against the handshake                                                                                      |
| Timing windows         | enforce <code>tWLMRD</code> , <code>tWLDQSEN</code> , <code>tWLO</code> and <code>tWLOE</code> as <b>runtime CSRs</b>                                                              |
| Result capture         | present the prime DQ bit’s reported value to the host, per chip select                                                                                                             |
| Telemetry              | <code>*_STATS</code> : attempts, flips observed, current window, and whether a leveling pass has ever completed                                                                    |

## 12.4 The timing windows

From JESD79-3F:

Table 3.4: Write-leveling timing windows

| Parameter | Value          | What the controller must do                                                                                                             |
|-----------|----------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| tWLDQSEN  | 25 nCK minimum | wait this long after entering leveling mode before driving DQS low / DQS# high; by its expiry the DRAM has applied ODT to those signals |
| tWLMRD    | 40 nCK minimum | wait this long before the first DQS pulse. <b>The maximum is controller-dependent</b> — the spec explicitly declines to bound it        |
| tWLO      | per speed bin  | the DRAM returns the result on the prime DQ bit(s) asynchronously, this long after the DQS edge                                         |
| tWLOE     | per speed bin  | DQ output uncertainty, defined to permit mismatch across DQ bits                                                                        |

**What the PHY does during leveling, and what it does not.** s7ddrphy exposes exactly `_wlevel_en` and `_wlevel_strobe`, and while leveling is enabled it forces `rddata_valid` on every phase (so the DRAM's answer returns through the ordinary read path) and forces `dqs_oe` (so DQS is driven continuously rather than gated per burst). It contains **no timeout of any kind** — which is why the maximum below is scoria's to define and not a number to be read off a PHY.

**Note on tWLMRD's open maximum.** Because JESD79-3F leaves the maximum to the controller, scoria must *define* one rather than inherit it, and the definition belongs in the CSR: a host that waits forever for a result it will never get is indistinguishable from a broken link. The requirement is a CSR-settable timeout with a distinct status bit, so a leveling attempt that never reports is reported as a timeout and not as a flip that failed to arrive.

**Note on tWLOE.** It exists to allow mismatch *across DQ bits*, which only matters if the controller samples more than the prime bit. If the implementation samples one bit, tWLOE is inert — and that should be an explicit, documented choice rather than an accident.

## 12.5 The telemetry requirement, and why it is not optional

---

The \*\_STATS surface is a requirement rather than a nicety, and the reason is a lesson paid for on pumice’s board: a detector that has never fired is not evidence of anything. pumice’s data-integrity CRC checker had never once fired, and proving it worked required deliberately corrupting the analog eye until it reported every beat mismatched.

A leveling interface that reports “converged” without exposing the attempts and the observed flips is the same trap. The requirement is therefore that a host can distinguish four states: never attempted, attempted and converged, attempted and timed out, and attempted with no flip found across the whole window.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

---

## 13 Refresh: DDR3 All-Bank and LPDDR3 Per-Bank

### 13.1 Marking: INHERITED, with a mode select

---

This chapter changed between v0.1 and v0.2 of this document. v0.1 marked `refresh_ctrl` MODIFIED on the assumption that LPDDR3 per-bank refresh was new work. It is not: **pumice already implements it**, for LPDDR2, and the mechanism LPDDR3 specifies is the same one.

### 13.2 What pumice already has

---

`refresh_ctrl` carries the retention interval, all-bank refresh, JEDEC’s plus-or-minus-eight postpone and pull-in scheduling — proven in formal across all sixteen postpone values — **and a REFpb bank rotor**.

The rotor is the part worth understanding, because it is not a scheduler:

`r_bank_rotor` advances exactly when a REFpb COMMAND is granted onto the wire,  
and HOLDS through REFab mode.

It mirrors the **device’s** internal bank counter. pumice’s own comment cites JESD209-2 §6.6 for why: the REFpb command *carries no bank address*. The controller cannot nominate a bank. It can

only track which bank the device will refresh next, so that the bank timers know which bank is about to become busy.

The rotor also deliberately holds across a mode change rather than resetting, because the device's counter persists — and clearing the mirror would desynchronise it from the device.

### 13.3 LPDDR3 keeps the same mechanism

---

JESD209-3C specifies a bank counter in the memory device, and states that the bank sequence for per-bank refresh is **fixed to be a sequential round-robin**. During a REFpb operation, banks other than the one being refreshed remain accessible.

That is the LPDDR2 mechanism unchanged, so the pumice implementation transfers.

**Important — this retires an open question.** v0.1 of this document asked (as Q5) whether the REFpb round-robin order should be strictly sequential or follow bank occupancy. **The question was malformed.** Occupancy-aware ordering is not available at any price: the command carries no bank address and the device's sequence is fixed. There is nothing to choose. The question is struck rather than deferred, because deferring it would imply a decision still exists.

### 13.4 Refresh by memtype

---

*Table 3.5: Refresh by memtype*

| Memtype | Refresh available                                                                                    |
|---------|------------------------------------------------------------------------------------------------------|
| DDR3    | all-bank REF only, with the JEDEC postpone window. DDR3 adds no controller-directed per-bank refresh |
| LPDDR3  | all-bank REF, plus REFpb in the device's fixed sequential order                                      |

Per-bank refresh matters because an all-bank refresh blocks every bank at once while REFpb leaves the others accessible. On a workload with bank parallelism that is the difference between a refresh being a stall and being nearly free.

The cleverer schemes — out-of-order per-bank refresh, write-refresh parallelisation, refresh pausing, SARP and DSARP — are LPDDR4/DDR5 or research, and scoria TASK-001 assigns them to the DDR4 area rather than here. That split stands.

### 13.5 Requirements

---

1. Offer all-bank refresh with the JEDEC postpone/pull-in window (inherited), and for LPDDR3 additionally REFpb with the rotor mirroring the device's counter (inherited).

2. The all-bank versus per-bank choice on LPDDR3 is a **runtime CSR**, so both can be measured from one bitstream. This is the only genuinely new surface in this block, and it is a mode select rather than a mechanism.
3. **Re-establish the retention property for the per-bank path.** The inherited formal proof covers retention headroom across every postpone value for all-bank refresh. REFpb changes the arithmetic that proof checks — the interval seen by any one bank is the all-bank interval multiplied by the bank count — so the property must be re-derived for the per-bank mode.
4. REFpb must respect per-bank timing as an ordinary occupant of that bank: a per-bank refresh is an operation on a bank and the bank timer owns it.

**Important:** requirement 3 is the one most likely to be skipped, and it is the reason refresh can be trusted at all. Carrying the all-bank property forward unchanged would prove the wrong thing while appearing green — the failure mode this family has been bitten by before.

**Note:** pumice’s idle detection is inherited with the block and is worth knowing about, because it looks like an arbitrary constant. Demand is CAM occupancy, which blinks off for a few cycles between bursts; treating those micro-gaps as idle would release postponed refreshes mid-stream and trigger pull-ins. Only a sustained gap counts, confirmed over sixteen cycles.

## 14 DFI v3.1 Master Interface, and the v2.1.1 Delta

### 14.1 The revision decision

---

scoria targets **DFI v3.1** (decision D1). pumice’s DFI layer was built to v2.1.1, so this chapter is the delta between them, measured across the two specifications: 126 distinct `dfi_*` signal names in v2.1.1, 151 in v3.1.

The decision was not obvious and the reasoning is on the record: v3.1’s *gain* for a DDR3/LPDDR3 controller is the leveling rework, the low-power split and `dfi_init`. Everything else it adds is DDR4/LPDDR4. Building to v2.1.1 and hand-adding write leveling would reuse pumice’s layer more directly. v3.1 wins because the per-CS leveling scheme is what the DDR4 controller will need too, and paying for it once here is cheaper than paying twice.

## 14.2 What does NOT change, and a diff that lies about it

**The datapath is inherited entirely.** One async-FIFO CDC, bubble-free, the unit of transfer a DFI word, DFI\_RATE = 4.

**Important.** A naive signal-name diff between the two specifications reports `dfi_address_p2`, `dfi_cs_n_p3`, `dfi_bank_n_p2`, `dfi_odt_p3` and their siblings as **removed in v3.1**. They are not removed. v2.1.1 enumerates phase variants explicitly; v3.1 generalises the notation to `_pN` — `dfi_address_pN`, `dfi_cs_n_pN`, `dfi_odt_pN`, `dfi_bank_pN` — and shows `_p0/_p1` only as examples. The 1:4 frequency ratio remains a defined mode (v3.1 §3.2.4, §3.3.4).

pumice runs four phases. An implementer who trusted that diff would rewrite a working, measured datapath for no reason, so the trap is recorded here rather than left for them to fall into.

## 14.3 What changes and matters

### 14.3.1 Leveling: reworked outright

*Table 4.1: The leveling interface, v2.1.1 against v3.1*

| v2.1.1                                                                                                                             | v3.1                                                                                                                               |
|------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <code>dfi_rdlvl_mode</code> , <code>dfi_rdlvl_load</code> ,<br><code>dfi_rdlvl_delay*</code> ,<br><code>dfi_rdlvl_gate_mode</code> | <b>removed</b>                                                                                                                     |
| <code>dfi_wrlvl_mode</code> , <code>dfi_wrlvl_load</code> ,<br><code>dfi_wrlvl_delay*</code>                                       | <b>removed</b>                                                                                                                     |
| <code>dfi_rdlvl_cs_n</code> , <code>dfi_wrlvl_cs_n</code><br>—                                                                     | <b>removed</b><br><code>dfi_phylvl_req_cs_n</code> ,<br><code>dfi_phylvl_ack_cs_n</code> (request/ack,<br>per chip select)         |
| —                                                                                                                                  | <code>dfi_phy_wrlvl_cs_n</code> ,<br><code>dfi_phy_rdlvl_cs_n</code> ,<br><code>dfi_phy_rdlvl_gate_cs_n</code> (which<br>leveling) |
| —                                                                                                                                  | <code>dfi_lvl_pattern</code> , <code>dfi_lvl_periodic</code>                                                                       |

This is the change scoria cannot route around, because DDR3 adds write leveling. `scoria_wrlvl_ifc` (Chapter 3.3) drives this interface, and because the search is firmware-driven the block is an interface and a set of timing windows rather than a state machine.

### 14.3.2 Low power: one request becomes two, and nothing consumes either

`dfi_lp_req` splits into `dfi_lp_ctrl_req` and `dfi_lp_data_req`, letting the command path and the data path be put to sleep independently.

**In practice, on this PHY family, the channel is unused.** `s7ddrphy` implements no DFI low-power interface, and LiteDRAM's generated DDR3 controller drives none — power-down there is CKE plus SRE/SRX. `scoria` exposes both requests for spec compliance and for a future PHY that consumes them, with the specified behaviour that an unacknowledged request times out and reports rather than blocking power-down. See Chapter 3.1.

### 14.3.3 Initialization and errors

`dfi_init` appears, giving initialization its own interface — which pairs with DDR3's RESET# pin. `dfi_parity_error` becomes `dfi_error` plus `dfi_error_info`, a wider surface that mostly carries DDR4 CA-parity detail; `scoria` uses the error indication and ignores the DDR4-specific encodings.

## 14.4 What `scoria` does NOT implement

Named explicitly so their absence is a decision on the record:

*Table 4.2: DFI v3.1 features deliberately unimplemented*

| Group                 | Signals                                                                                        | Why not                                                   |
|-----------------------|------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| DDR4 command encoding | <code>dfi_act_n*</code> , <code>dfi_bg*</code> ,<br><code>dfi_cid*</code>                      | DDR3 has no dedicated ACT pin, no bank groups, no chip ID |
| CA parity, CRC, alert | <code>dfi_alert_n*</code> ,<br><code>dfi_parity_in_p*</code>                                   | DDR4-era                                                  |
| Data Bus Inversion    | <code>dfi_rddata_dbi*</code>                                                                   | DDR4/LPDDR4                                               |
| CA training           | <code>dfi_calvl_*</code> ,<br><code>dfi_ca_capture</code> ,<br><code>dfi_phy_calvl_cs_n</code> | LPDDR4-era                                                |

That is the majority of v3.1's growth over v2.1.1. Targeting the newer revision does not mean implementing all of it.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

## 15 AXI4 Slave and APB CSR

Both interfaces are **INHERITED** from pumice without functional change. This chapter records what that inheritance includes, so that “inherited” is not mistaken for “unspecified”.

### 15.1 AXI4 slave

The host-side interface is pumice’s `axi4_ifc`: one read intake and one write intake at 1:1, a write-data CAM, a read-reorder CAM, in-order commit, and FSM-free split and aggregate where the aggregate and last flags ride with the command in the CAM.

Nothing about DDR3 or LPDDR3 reaches the host side. The DRAM generation is invisible across this boundary, which is the property that makes the family worth having.

**Width gearing is inherited too.** The host bus width need not match the DRAM burst width: formally-verified `dwidth` converters wrap the core, so the host side is free. This is why `scoria_top_geared` exists as a separate top alongside `scoria_top`.

### 15.2 APB CSR

A PeakRDL-generated register block behind an APB slave. The register *set* grows (Chapter 5), but the interface and the generation flow do not change.

**Important — the regeneration rule applies unchanged.** The register block is generated through `bin/peakrdl_generate.py`, never `raw peakrdl`, and always with an explicit `-o`. The wrapper emits the RTL, the documentation and the Python regmap together; `raw peakrdl` skips the latter two and the default output path can write an orphan copy the build never reads, which then drifts.

**Important — registers are accessed by NAME.** Host code reads and writes through the generated regmap, never by hardcoded offset. Offsets churn whenever the RDL changes, and a host that hardcodes them fails silently by hitting a neighbouring register that happens to decode.

### 15.3 The CDC on the CSR path, and why it is an async FIFO

If scoria is integrated the way pumice is on its board — an APB window on one reset domain reaching a core on another — then the CDC between them must be a gray-pointer async FIFO, not a toggle-parity handshake.

The reason is an asymmetric reset, and it is worth carrying forward because it cost real debugging on pumice: the APB side runs on `presetn` and the core side on `aresetn`, and a soft reset pulses **only the core side**. A toggle-parity handshake desynchronises across that and leaves the response stream permanently offset by one transaction — every readback returns the answer to the previous question, which looks like a lagging register file rather than a CDC fault.

This is a note about integration rather than about scoria’s own ports, but it belongs here because the next person to simplify that CDC will be reading this chapter.

## 16 scoria\_pkg, and Build-Time vs Runtime

### 16.1 The package (decision D3)

---

scoria gets its own `scoria_pkg`, with a one-bit memtype enum:

```
typedef enum logic {  
    MEMTYPE_DDR3    = 1'b0,  
    MEMTYPE_LPDDR3  = 1'b1  
} memtype_e;
```

It does **not** extend `pumice_pkg`. That package’s `memtype_e` is one bit wide and already spent on {DDR2, LPDDR2}; adding two more members means widening the field, which reaches pumice’s RTL and its CSR — a measured, shipping design — for the benefit of a controller with no RTL. The trade is wrong in that direction.

**When this changes.** At the start of the DDR4/LPDDR4 controller, a shared `mem_ctrl_pkg` with a two-bit memtype becomes worth the migration, and both existing controllers move to it in the same pass. That is the right moment because the cost is paid once with three members to amortise it.

**Note:** until then `scoria_pkg` and `pumice_pkg` carry near-identical type definitions. The duplication is deliberate and time-boxed. It is recorded here so that a later reader does not “fix” it by widening pumice’s CSR — which is precisely the thing this decision avoids.

**Inherited from pumice’s package:** the row field is already 18 bits wide, with the comment “DDR3 forward-compat”. That was written in anticipation of this controller and carries over as-is.

### 16.2 The rule: timings are runtime, geometry is build-time

---

Inherited from pumice and restated because it is the single most load-bearing convention in the family:

**Every enforced timing is a runtime CSR.** Not a parameter. A timing compiled in cannot be swept, and a controller whose timings cannot be swept cannot be characterized — which is how a controller ends up with numbers nobody can explain.

**Build-time is reserved for what cannot be runtime:** bus widths, the DFI gear ratio, CAM depths, bank and rank counts. The test is whether changing it changes the amount of hardware.

**As of v0.5 these have values,** fixed by the target design point in Chapter 2.4: DFI\_RATE = 4 (the PHY's nphases), NUM\_BANKS = 8, row width 15, column width 10, DQ width 32, NUM\_RANKS = 1. The DDR3-800 JEDEC timing set for the part is tabulated there too, and those remain runtime CSRs initialized to those values.

*Table 5.1: Build-time versus runtime*

| Kind             | Build-time or runtime | Examples                                                     |
|------------------|-----------------------|--------------------------------------------------------------|
| Bus and geometry | build                 | data width, address width, NUM_BANKS, NUM_RANKS, CAM depths  |
| DFI gearing      | build                 | DFI_RATE, which must equal the PHY's phase count             |
| Memtype          | build                 | MEMTYPE_DDR3 / MEMTYPE_LPDDR3                                |
| JEDEC timings    | runtime               | every command-spacing and recovery parameter                 |
| New DDR3 timings | runtime               | tZQinit, tZQoper, tZQCS, tWLMRD, tWLDQSEN, tWLO, tWLOE       |
| Policy selects   | runtime               | page policy, refresh mode (all-bank vs REFpb), ZQCS interval |

**Important — DFI\_RATE must equal the PHY's phase count, set identically on both sides.** This is inherited and it is not negotiable: a mismatch is not a performance problem, it is a functional break, and it is the kind that presents as data corruption rather than as an error.

## 16.3 New CSR groups

The register set grows by three groups. Exact offsets are not fixed in this edition — they come from the RDL, which is authored with the RTL — but the content is specified:

*Table 5.2: New CSR groups*

| Group | Contents                          |
|-------|-----------------------------------|
| ZQ    | tZQinit, tZQoper, tZQCS, the ZQCS |

| Group          | Contents                                                                                                                                  |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------|
|                | interval, and a count of calibrations issued                                                                                              |
| Write leveling | tWLMRD (with its controller-defined maximum as a timeout), tWLDQSEN, tWLO, tWLOE, per-CS result, and the *_STATS telemetry of Chapter 3.3 |
| Refresh mode   | all-bank versus REFpb select, for LPDDR3                                                                                                  |

**Note:** tWLMRD’s maximum is a *scoria* number, not a JEDEC one — JESD79-3F declares it controller-dependent. It appears in the CSR as a timeout with its own status bit, so that a leveling attempt which never reports is distinguishable from one that reports no flip.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

## 17 Verification Strategy, and the Open Questions

### 17.1 The target is the DFI boundary

Verification is against the DV repository’s DFI bus functional model, in cocotb. No board is required to verify *scoria*, and that has not changed.

**A target board is now named, though** (Sean, 2026-09-30, Chapter 2.4): the Genesys 2 with K7DDRPHY and two MT41J256M16 devices at DDR3-800. That fixes the parameters the DUT is verified *with*, so the BFM should be configured to that geometry – 8 banks, 32768 rows, 1024 columns, a 32-bit bus, four DFI phases – rather than to an arbitrary one. Verifying at the geometry the design point uses is free, and verifying at a different one silently weakens every result.

That boundary is chosen for the same reason pumice chose it: it is the one interface where the controller’s obligations are fully specified by a standard, so a model can be authoritative rather than approximate. pumice’s experience is that code passing against the BFM passes on hardware modulo PHY training — which is exactly the residue this specification pushes into firmware anyway.

## 17.2 What must be verified, beyond inheriting pumice's suites

The inherited blocks bring their tests with them. The new and modified ones need new coverage, and four items are specific enough to name now:

1. **The init sequence, in JEDEC's order.** Not “the DRAM came up” but step-by-step: RESET# held, the 500 us wait, clocks stable before CKE, CKE continuously high, tXPR before the first MRS, then MR2, MR3, MR1, MR0, then ZQCL, then tDLLK and tZQinit. A checker that asserts the *order*, because order is what pumice got wrong on its equivalent.
2. **ODT during init.** High-impedance while RESET# is asserted, and held statically — LOW if RTT\_NOM is enabled in MR1 — until init completes.
3. **Write leveling as a protocol, not an outcome.** The handshake sequence and every timing window, with the twLMRD timeout exercised deliberately. A leveling interface whose timeout path has never run is an untested path on the only path that reports failure.
4. **Per-bank refresh retention.** The inherited formal property proves retention headroom across all sixteen postpone values for all-bank refresh. REFpb changes that arithmetic — the per-bank interval is the all-bank interval times the bank count — so the property must be re-established, not carried forward. Carried forward unchanged it would prove the wrong thing and appear green.

**Important — the formal area is the right home for the spacing properties.** No assertions go in the RTL. pumice's formal/pumice area proved nine modules and found two real bugs, including the global\_timers next-state defect this specification tells scoria to inherit the fix for. scoria should open its own formal area early, and the cmd\_history\_checker — which independently re-derives JEDEC spacing from the issued command stream — should grow DDR3's parameters before the RTL is trusted.

## 17.3 Question resolutions (v0.2)

Five questions were listed in v0.1. Two are answered, one is struck as malformed, and two are deferred with a named condition.

Table 6.1: Question resolutions

| #  | Question                                 | Resolution                                                          |
|----|------------------------------------------|---------------------------------------------------------------------|
| Q1 | DFI low-power handshake ordering on exit | <b>ANSWERED: the question does not arise on a Series-7 PHY.</b> See |

| #  | Question                                         | Resolution                                                                                       |
|----|--------------------------------------------------|--------------------------------------------------------------------------------------------------|
|    |                                                  | below                                                                                            |
| Q2 | May periodic ZQCS preempt queued demand traffic? | <b>ANSWERED: no. Request/grant, sharing the maintenance sequencer</b>                            |
| Q3 | Sample one prime DQ bit or several?              | <b>ANSWERED: the prime bit only</b> , which makes tWLOE inert                                    |
| Q4 | The value of tWLMRD's controller-defined maximum | <b>ANSWERED in kind: no PHY supplies one.</b> It is a policy value, not a measurement. See below |
| Q5 | REFpb ordering: sequential or occupancy-aware?   | <b>STRUCK — malformed.</b> The device owns the sequence (Ch 3.4)                                 |

### 17.3.1 Q1 and Q4: I was wrong to call these PHY-blocked

Both were deferred in v0.2 and v0.3 “pending a PHY choice”. Examining the PHY shows that framing was wrong in both cases, and the correction matters because it unblocks the last two questions in this document.

**The PHY choice is narrower than it looked.** LiteDRAM's Series-7 PHY is one module, `s7ddrphy`, whose `A7DDDRPHY` and `K7DDDRPHY` classes differ by target family, not by DFI behaviour, and it supports DDR2 and DDR3 with write leveling. For any Xilinx 7-series target – which is every board this repository owns – the PHY is that module. There was no branch to wait on.

**Q4: no PHY supplies a tWLMRD maximum, because none can.** `s7ddrphy` exposes exactly two leveling controls, `_wlevel_en` (a CSR storage field) and `_wlevel_strobe` (a CSR pulse), and contains **no timeout of any kind**. JESD79-3F declares the maximum controller-dependent and the PHY declines it too, so the value is a *policy* decision — how long to wait before declaring a leveling attempt failed — not a number to be measured off a PHY. *scoria* defines it, as this document already specified: a CSR timeout with its own status bit. Choosing the number is a one-line decision, not an experiment.

**Q1: the question does not arise on this PHY.** `s7ddrphy` implements **no DFI low-power interface at all** — no `lp_req`, no `lp_ack`, nothing. Nor does LiteDRAM's generated DDR3 controller: a search of the generated core for `dfi_lp` returns zero hits, while `CKE` appears 92 times. Power-down on this PHY family is achieved by **DRAM command** — `CKE` plus `SRE/SRX` — and not through the DFI low-power channel.

So there is no exit-ordering handshake to get wrong, because there is no counterparty to handshake with. The requirement changes accordingly:

- `powerdown_ctrl` achieves precharge-power-down, active-power-down and self-refresh through CKE and SRE/SRX, which is what pumice already does.
- scoria still *exposes* `dfi_lp_ctrl_req` and `dfi_lp_data_req`, because DFI v3.1 defines them and a future PHY may consume them. Their specified behaviour when no acknowledgement ever arrives is to **time out and report**, never to block power-down — an unacknowledged request must not be able to wedge the controller.
- The ordering question returns only with a PHY that implements the interface, and at that point it is answerable by reading that PHY rather than by speculation.

**Note on the correction.** This is the second framing error in this document's short life: v0.1 asked Q5, which the DRAM standard had already settled, and v0.2/v0.3 deferred Q1 and Q4 to a decision that turned out not to exist. Both came from the same habit — treating something as an open choice without first checking whether the thing it depends on had already answered it. The cheap check is to read the artifact.

### 17.3.2 Q2: ZQCS is a request, never a preemption

**Answered from a DDR3 LiteDRAM core, generated for this purpose.** v0.2 answered this by analogy, because the LiteDRAM core in the pumice flow is a DDR2 configuration and contains no ZQ logic at all. Sean's suggestion was to generate a DDR3 one; that core now exists and the analogy is no longer needed.

What the DDR3 core does (`bin/litedram_ddr3_ref.yml` regenerates it):

- **ZQCS shares the refresher's FSM.** There is one refresher state machine, and it starts the ZQCS executer from inside itself and waits for its done. Maintenance is not a separate agent competing for the bus; it is one sequencer issuing both kinds of maintenance command.
- **It requests, and waits.** `refresh_req` to every bank machine is driven from the maintenance command's valid. The bank machines must grant – drain and precharge – before the command issues. Nothing is preempted.
- **The executer is a timed sequence,** a trigger counter stepping to 19 and then asserting done, with the periodic timer held off (`zqcs_timer_wait`) until it completes. So  $t_{ZQCS}$  is enforced by holding the *next* interval, not by blocking traffic.

That is three independent controllers agreeing – LiteDRAM DDR2 refresh, LiteDRAM DDR3 ZQCS, and pumice refresh – so the baseline is settled rather than merely plausible: `scoria_zq_ctrl` raises a demand and waits for a grant.

**Design consequence worth taking from LiteDRAM:** it shares one sequencer between refresh and ZQ rather than giving ZQ its own. `scoria`’s Chapter 2.3 specifies `scoria_zq_ctrl` as a separate FUB, and that remains the plan – pumice’s `refresh_ctrl` is already substantial and adding a second maintenance mode to it would couple two independently-verifiable things. But the *interface* both present to the scheduler should be identical, so that the arbiter sees one kind of maintenance demand with a source tag rather than two special cases.

**Condition to revisit unchanged:** if characterization shows ZQCS starved under sustained demand – the interval slipping materially past its CSR value – placement becomes TASK-001’s scheduling question. The issued-calibration counter required in Chapter 3.2 is what makes that measurable rather than suspected.

### 17.3.3 Q3: sample the prime DQ bit only

The write-leveling interface samples the prime DQ bit and not the full byte. The consequence is explicit: **twLOE is inert in this implementation.** It exists in JESD79-3F to allow mismatch *across* DQ bits, which only matters to a controller sampling several.

twLOE remains a CSR field so that a later implementation can sample wider without a register-map change, and it is documented as unused rather than omitted — an absent register is a silent limitation, while a present one documented as inert is a decision a reader can find.

### 17.3.4 Q5: struck rather than deferred

v0.1 asked whether REFpb round-robin should be sequential or occupancy-aware. There is nothing to choose: the REFpb command carries no bank address, the sequence is fixed in the device, and the controller’s rotor only mirrors the device’s counter. JESD209-3C states the sequence is fixed sequential round-robin; pumice’s implementation already cites JESD209-2 §6.6 for the same property in LPDDR2.

It is struck rather than deferred because deferring implies a decision still exists. This one never did, and the correction also moved `refresh_ctrl` from MODIFIED to INHERITED — see Chapter 3.4.

## 17.4 What would make this a 1.0

---

Three things, in order:

1. A value chosen for twLMRD’s maximum – Q4 reduced this to a one-line policy decision, and with CK at 2.5 ns on the named target a generous bound is easy to pick.

2. The RTL written, and this document reconciled against it — with every INHERITED marking either confirmed or corrected. A marking that turns out to be wrong is a defect in this document, and the correction goes here rather than being dropped silently.
3. The exact CSR map, which comes from the RDL and cannot honestly precede it.

Until then this is a 0.1: a specification good enough to implement against, and explicit about where it is not yet a description of anything.